Transformer architectures

Intro to ML from an LLM standpoint

In this long read we'll discuss some of the milestones of transformer architecture development that shaped how today's LLM are built.

1 MLP (FFN)

A feedforward block in a transformer is often a two-layer MLP which originally had ReLU activation:

$$\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1 + b_1)W_2 + b_2$$

Several improvements were suggested.

1.1 No bias

Yes, let's just get rid of b_1 and b_2. Seems to increase training stability.

1.2 Activation functions

Idea: replace $\mathrm{ReLU}(x) = \max(0, x)$ by something smooth.

A well known substitute is

$$\mathrm{ELU}(x) = \begin{cases} x, & x > 0 \\ a(e^x - 1), & x \leqslant 0 \end{cases}$$

with some $a > 0$, but several other functions proved better.

1. GELU introduced in this paper is

$$\mathrm{GELU}(x) = x\mathbb{P}\{\xi \leqslant x\} = x\Phi(X),$$

where $\xi \sim \mathcal{N}(0, 1)$ and Φ is the cdf of a standard gaussian distribution.

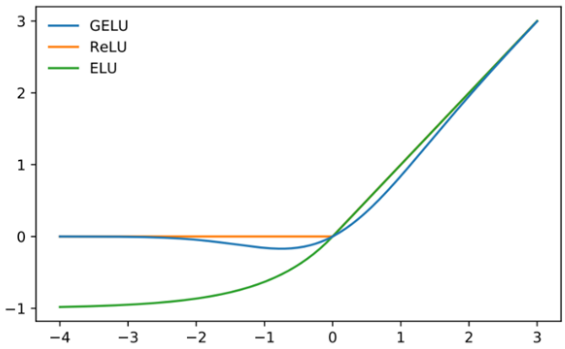


Figure 1: The GELU ($\mu = 0, \sigma = 1$), ReLU, and ELU ($\alpha = 1$).

2. **Swish** introduced in Searching for activation functions is

$$\mathrm{Swish}(x) = x \cdot \sigma(\beta x),$$

where β is a parameter and σ is the familiar sigmoid function. GELU can be approximated with Swish as $x\sigma(1.702x)$.

3. **SwiGLU** was introduced in GLU Variants Improve Transformer. It's actually more than just a new activation, because it also adds third trainable weight matrix to the feedforward block:

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x, W_1, V, W_2) = (\mathrm{Swish}_{\beta=1}(xW_1) \otimes xV)W_2,$$

where $\otimes$ stands for elementwise product.

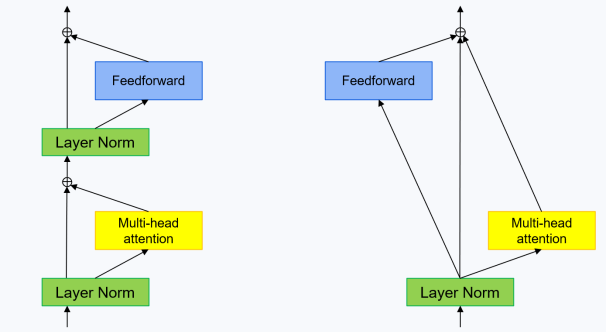
SwiGLU reminds a gating mechanism from LSTMs: it's like xV is the information and $\mathrm{Swish}_{\beta=1}(xW_1)$ controls how much of this information we want to

pass through the FFN layer. Not exactly like that, of course (because Swish is not σ), but this may help with the intuition.

1.3 "Parallel" formulation

Although this idea is about the whole transformer block architecture, I'll mention it in this section.

It is suggested by the authors of mesh-transformer-jax and later used in PaLM. In a usual transformer block attention and MLP are arranged consequently, and this approach decouples them:



Note the peculiar position of LayerNorm. You'll see more about this in the following section.

2 Normalization

2.1 RMSNorm

Root Mean Square Layer Normalization paper

This normalization technique ignores centering and just fixes scale. So, for a vector a the new values will be:

$$\overline{a}_i = \frac{a_i}{\mathrm{RMS}(a_i)} g_i,$$

where g_i is trainable and

$$\text{RMS}(a) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2}$$

RMSNorm yields comparable performance against LayerNorm but shows superiority in terms of running speed with a speed-up of $7\% \sim 64\%$. It seems to be state-of-the-art now.

2.2 Pre-normalization

Introduced in this paper, suggests to

- perform normalization before attention and before FFN rather than after them;
- run normalization in parallel to the residue stream.

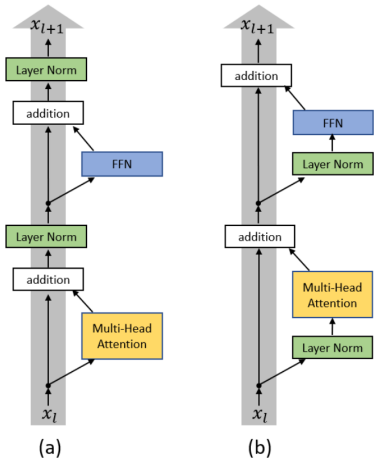


Figure 1. (a) Post-LN Transformer layer; (b) Pre-LN Transformer layer.

Motivation: authors show that with Post-LN the expected gradients of the parameters near the output layer are large. Therefore, warm-up stage is needed in the beginning of training process where the optimization starts with an extremely small learning rate. Otherwise the optimization process can become unstable. Check theoretical analysis in the paper.

3 Some additional attention details

3.1 Masked attention

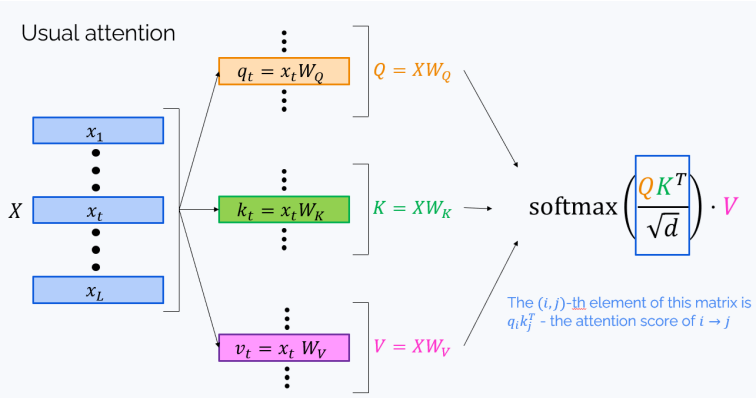
There is an important difference between how self-attention is used in encoder-only models (such as BERT) and in LLMs (which are decoder-only models):

- In encoder-only models, each token "attends" to each token.
- In LLMs, a token never "attends" to future tokens. Indeed, when we're generating a text, we can't know what's in the future.

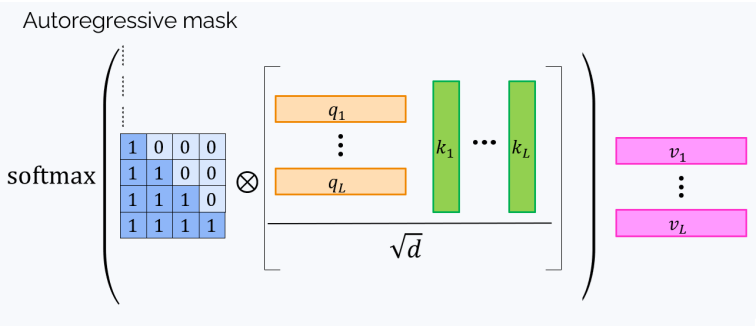
At this point, it's good to note that there are in a sense two different modes of LLM operation:

- During the **autoregressive generation** phase we generate new tokens one by one, and these tokens just can't "see" whatever is not yet generated.
- However, during the **prompt processing** phase the LLM has access to all the prompt tokens, and the prompt tokens must be deliberately prohibited from "looking" at the later tokens. This might have been implemented by processing the prompt token by token, but this would be very inefficient. Instead, the prompt is processed in one pass with **attention masking**.

To understand what the attention mask is, let's recall how self-attention works:



To prevent "looking into the future", we need to zero all elements of QK^T over the main diagonal (those with $i < j$). It can be done by elementwise multiplication on the $0 - 1$ matrix called **attention mask**:



Here, $\otimes$ stands for the elementwise product. The final formula for the attention is:

$$o = \textbf{softmax}\left(\frac{M \otimes QK^T}{\sqrt{d}}\right) V$$

Note. It's not often mentioned, but in today's LLMs the self-attention layer often has one additional operation:

$$y = oW_o$$

Note. In today's implementations elementwise product is often replaced by elementwise sum with a matrix M'

that has $-\infty$ above the diagonal. Softmax of $-\infty$ is zero, so that also works.

3.2 Some details about attention heads

Despite being described as totally separate layers, in reality attention heads work as follows:

- Theoretically, each i-th attention head has its own matrices $W_{i,Q}, W_{i,K}, W_{i,V}, W_{i,O}$,
- In practice, they are stored and applied each as one matrix:

  $$W_Q = \begin{pmatrix} W_{1,Q} & W_{2,Q} & \cdots & W_{\text{n\_heads},Q} \end{pmatrix}$$

  etc. When applying it, we do

  $$q_{total} = xW_Q,$$

  and after that the row vector q_{total} is cut into n_heads row vectors $q_1, q_2, \ldots, q_{\text{n\_heads}}$.
- So, for example, if Llama3-8B has model dimension (=hidden size) 4,096 and 32 attention heads, then each attention head's query has dimension $\frac{4096}{32} = 128$.
- I don't mention Llama's keys and values, because there are some additional considerations; see the Group Query Attention section.

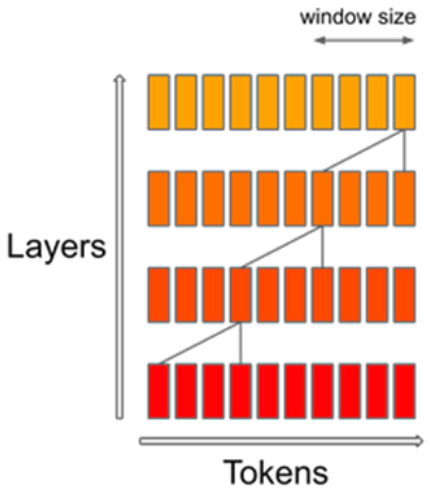
4 The story of attention

Attention mechanism is the heart of every transformer, and it's not surprising that many variations of it emerged since 2017.

The main problem with attention is its quadratic complexity. Indeed, each token must attend to each token. Moreover, standard procedure involves calculating the matrix QK^T (product of matrix of all queries and the matrix of all keys) which can be very large. Complexity of attention mechanism is one of the main bottlenecks in the struggle towards large context length, and many of the improvements below aim towards making attention al little more lightweight.

4.1 Sliding window attention

Probably drawing inspiration from convolutional networks, it suggests to consider only a fixed window around each token thus making the computational cost linear on the context length.



The upper layers receive information from longer subsequence. As in convolutions, attention window can be dilated to make receptive field wider.

Though Sliding Window Attention is not used in top-tier LLMs, it's a popular ingredient in the attempts at making LLMs more efficient with long context.

4.2 Group Query Attention

Even a single attention mechanism is costly, and going multi-head only makes it worse. An idea to mitigate this: have only one attention head for values and keys, still many heads for queries. It's called **multi-query attention**, and it's nice, but very restrictive.

The next step was suggested in GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints: to have many value heads which are grouped on several value and key heads:

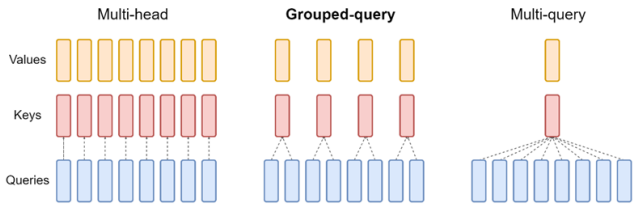


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

Now we can return to Llama-3. If we look at the technical report, it shows that "Key/Value Heads" is 8. Given that there are 32 attention heads (=how many queries), we see that there are 4 query heads per key/value. Moreover, with attention head dimension 128, we may see that the dimensions of the total W_Q and W_K are

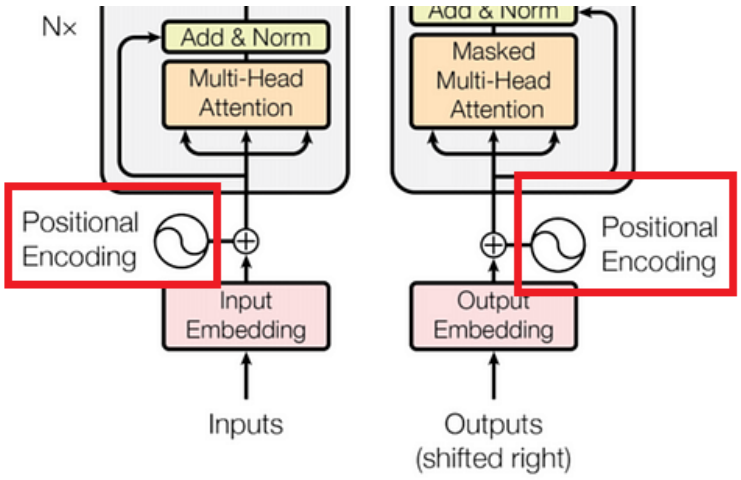
$$\text{hidden\_dim} \times (128 \cdot 8) = 4096 \times 1024.$$

4.3 Key-Value Cache

It is now a natural feature of almost every transformer model. It stores keys and values (that is $k_i = x_i W_K$ and $v_j = x_j W_V$) allowing us not to recompute them every time. However it has its own drawback: memory consumption. For long sequences, the size of a KV-cache may become comparable with the size of the model itself.

5 Positional encoding

Attention mechanism is cool, but it doesn't account to the ordering of tokens. To add this information, the original Transformer paper suggested absolute positional encoding: a special vector for each position number i that is added to the token embedding.



In this section, we'll discuss how we can improve this mechanism.

5.1 Relative positional encoding

Self-Attention with Relative Position Representations

Motivation: for the attention mechanism information about relative positioning is more relevant than about absolute positions.

First of all, let's change our view on positional embeddings. They are needed to introduce information about positions into the attention mechanism, so let's consider them as details inside it:

Without positional encoding

$$z_i = \sum_{j=1}^{n} \alpha_{ij} x_j W_V$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{n} \exp e_{ik}}$$

$$e_{ij} = \frac{(x_i W_Q)(x_j W_K)^T}{\sqrt{d}}$$

With positional encoding

$$z_i = \sum_{j=1}^{n} \alpha_{ij} \left(x_j W_V + \pi_{ij}^V\right)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{n} \exp e_{ik}}$$

$$e_{ij} = \frac{(x_i W_Q)(x_j W_K + \pi_{ij}^V)^T}{\sqrt{d}}$$

Note that in the original Transformer architecture positional embeddings are indeed introduced into each attention layer due to residual connections. However, in modern architectures it is no longer equivalent due to prenormalization.

Now, relative positional encoding suggests to set

$$\pi_{ij}^K = w^K_{\text{clip}(j-i,k)},$$

$$\pi_{ij}^V = w^V_{\text{clip}(j-i,k)},$$

where $\text{clip}(x, k) = \max(-k, \min(k, x))$ for some fixed clipping distance k (the width of an attention window). Vectors w^K and w^V are learnt during the traning of a transformer.

5.2 Rotary embeddings (RoPE)

They were introduced in the RoFormer paper and have since become a state of the art in the world of position encoding.

Again, we consider positional encoding as a way of influencing attention mechanism.

The idea is often formulated in the language of complex numbers, and I'm explaining the concept shortly in 7[the

appendix]. But here, I will use the language of matrices instead.

Rotation matrix in 2d. Counterclockwise rotation by angle φ around the origin is a linear transformation of the 2d plane $\mathbb{R}^2$. As such, it can be written using a matrix

$$(x, y) \mapsto (x, y) \cdot \begin{pmatrix} \cos\varphi & \sin\varphi \\ -\sin\varphi & \cos\varphi \end{pmatrix}$$

It's cumbersome to write this matrix every time we need to rotate something. Luckily, we can use a shortcut:

$$(x, y) \cdot e^{i\varphi} \textbf{ is the same as } (x, y) \cdot \begin{pmatrix} \cos\varphi & \sin\varphi \\ -\sin\varphi & \cos\varphi \end{pmatrix}$$

I very briefly explain the math behind $e^{i\varphi}$ in the 7appendix, but for the purpose of understanding the positional embedding, you can just treat it as a short notation for the rotation matrix.

Now, back to rotary embeddings!

The authors draw inspiration in yet another look of additive positional encoding for queries and keys:

$$q_m = (x_m + \pi_m^Q)W_Q, \quad k_n = (x_n + \pi_n^K)W_K$$

But instead of using the ready formulas, they want to substitute the right parts with some new functions

$$q_m = f_q(x_m, m), \quad k_n = f_k(x_n, n)$$

such that $q_m k_n^T$ only depends on the relative position $m-n$ (and not on m and n themselves).

For 2-dimensional x_i, k_i, q_i the authors prove mathematically that f_q and f_k should be of the form:

$$f_q(x_m, m) = x_m W_Q e^{im\theta}, \quad f_k(x_n, n) = x_n W_K e^{in\theta}.$$

Indeed, in this case

$$f_q(x_m, m) \cdot f_k(x_n, n)^T =$$

$$= x_m W_Q \begin{pmatrix} \cos m\theta & \sin m\theta \\ -\sin m\theta & \cos m\theta \end{pmatrix} \cdot \begin{pmatrix} \cos n\theta & -\sin n\theta \\ \sin n\theta & \cos n\theta \end{pmatrix} W_K^T x_n^T =$$

$$x_m W_Q e^{i(m-n)\theta} W_K^T x_n^T$$

For arbitrary dimension d (which we assume even) we use a block-diagonal rotation matrix:

$$f_q(x_m, m) = x_m W_Q R^d_{\Theta,m}, \quad f_k(x_n, n) = x_n W_K R^d_{\Theta,n},$$

where

$$R^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & \sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ -\sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & \sin m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & -\sin m\theta_2 & \cos m\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_{d/2} & \sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \dots & -\sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix},$$

where the parameters Θ are set to

$$\theta_i = 10000^{-2(i-1)/d}$$

Be cautious! If you browse through the original paper, you'll find that everything is multiplied in the inverse order, for example, $f_q(x_m, m) = R^d_{\Theta,m} W_Q x_m$. The matrix $R^d_{\Theta,m}$ also appears transpose. The cause of this ambiguity is the fact that ML-community can never agree on a simple question: *are x_i row vectors or column vectors?* In our interpretation x_i are *rows*, so they get multiplied from the right. The authors of RoPE paper see x_i as columns, so all the multipliers go to the left.

Long-term decay. RoPE embeddings provide long-term decay property which means the $q_m k_n^T$ will decay

when the relative position increases. This is an illustration from the paper:

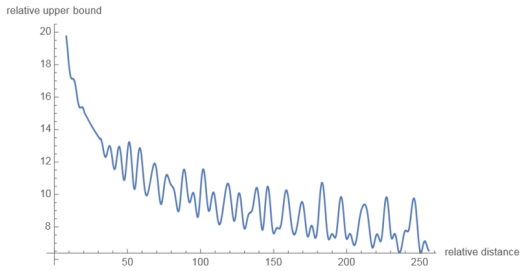


Figure 2: Long-term decay of RoPE.

Linearized attention. Actually, in the RoPE paper the formulation of attention mechanism is a bit peculiar, inspired by Transformers are RNNs paper. Let's dive into it. The usual mechanism having queries q_n, keys k_n and values v_n transforms values as:

$$v'_n = \frac{\sum_{m=1}^{N} \mathrm{sim}(q_n, k_m) v_m}{\sum_{m=1}^{N} \mathrm{sim}(q_n, k_m)},$$

where sim is a similarity function of sorts, for example, $\mathrm{sim}(q_n, k_m) = \exp\left(\frac{q_m k_n^T}{\sqrt{d}}\right)$. Indeed, the traditional

$$v'_n = \sum_{m=1}^{N} \frac{\exp\left(\frac{q_m k_n^T}{\sqrt{d}}\right)}{\sum_{m=1}^{N} \exp\left(\frac{q_m k_n^T}{\sqrt{d}}\right)} v_m,$$

can be rewritten exactly like this.

The authors of Transformers are RNNs observe that we can choose different similarity functions as long as they are positive. They suggest taking

$$\mathrm{sim}(q_n, k_m) = \psi(q_n)\phi(k_m)^T$$

for some functions ψ, ϕ. In particular, they propose $\psi(x) =$

$\phi(x) = \mathrm{ELU}(x) + 1$, where

$$\mathrm{ELU}(x) = \begin{cases} x, & x > 0 \\ a(e^x - 1), & x \leqslant 0 \end{cases}$$

In their turn, the authors of RoPE suggest taking the formula

$$v'_n = \sum_{m=1}^{N} \frac{\psi(q_n)\phi(k_m)^T}{\sum_{m=1}^{N} \psi(q_n)\phi(k_m)^T} v_m,$$

and plugging in RoPE in the following way:

$$v'_n = \sum_{m=1}^{N} \frac{\left(\psi(q_n)R^d_{\Theta,n}\right)\left(\phi(k_m)R^d_{\Theta,m}\right)^T}{\sum_{m=1}^{N} \psi(q_n)\phi(k_m)^T} v_m,$$

Note that there are no RoPEs in the denominator (to avoid it becoming zero), so the numbers

$$\frac{\left(\psi(q_n)R^d_{\Theta,n}\right)\left(\phi(k_m)R^d_{\Theta,m}\right)^T}{\sum_{m=1}^{N} \psi(q_n)\phi(k_m)^T}, m = 1, \ldots, N$$

no longer give a probabilistic distribution, but the authors claim that it's all right anyway.

6 Mixture of experts

This approach is featured in the Mixtral model, and we realy recommend to browse the Mixture of Experts Explained blog post by the Mixtral to better understand its internals and take a glimpse on how to make it really efficient.

Mixtral has a similar architecture to Mistral 7B, but some of the Feedforward layers are replaced with a sparse MoE (Mixture of Experts) layer.

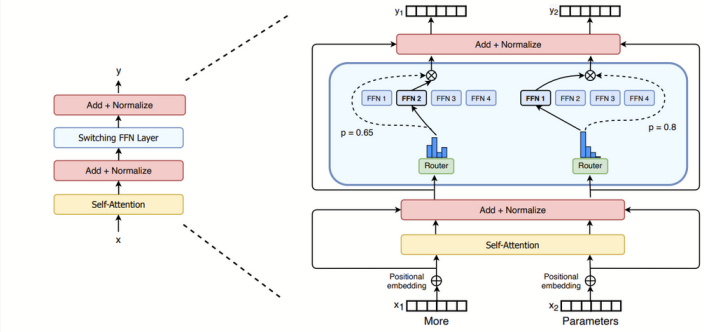


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = "More" and x_2 = "Parameters" below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

What we have here:

- Gating function that decides which expert will receive the input x. It works as follows. First of all, for each expert number i it calculates

  $$H(x)_i = (x \cdot W_g)_i + \text{StandardNormal}() \cdot \text{SoftPlus}((x \cdot W_{noise})_i),$$

  where W's are trainable weigths and randomness aids load balancing. Then, we only pick experts with top k values of H:

  $$\text{KeepTopK}(H(x), k)_i = \begin{cases} H(x)_i, & \text{if } H(x)_i \text{ is among the top-}k \text{ of } H(x)_i, \\ -\infty, & \text{otherwise.} \end{cases},$$

  And finally

  $$G(x) = \text{softmax}(\text{KeepTopK}(H(x), k))$$

- Several experts E_i, k of which are employed each time:

  $$y = \sum_i G(x)_i E_i(x)$$

Why does it matter? If we compare Mistral to Mixtral, we see two things:

- Mixtral has much more total parameters: 46.7B as opposed to 7B of Mistral. So, you'll need more memory to store it, but in the same time the model is potentially much more powerful than Mistral.
- However, on inference time you don't use all the parameters, because out of many experts you only employ several (for example, 2). In case of Mixtral, only 12.9B parameters are used at each inference call. This makes a model almost as quick as Mistral, while being much more powerful.

There are many more interesting features for explore in this approach, for example, load balancing. Dealing with experts is worth it if you can parallelize their job, but depending on gating some of them may remain underemployed crippling overall efficiency. The authors use some interesting ideas for that.

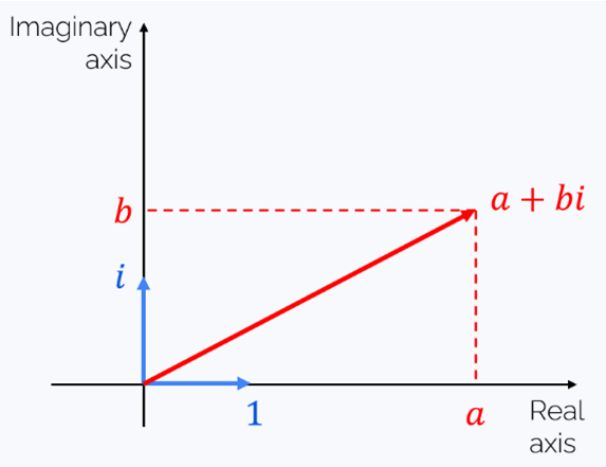
7 Appendix: complex numbers

The idea is usually formulated in the language of complex numbers, so let's remind what's that. In essence, a complex number is something of form $a + bi$, where i is a phantom number satisfying $i^2 = -1$. There are several good things about complex numbers:

- A sum of two complex numbers is a complex number:

$$(a_1 + b_1 i) + (a_2 + b_2 i) = (a_1 + a_2) + (b_1 + b_2)i$$

- Complex numbers have a convenient interpretation as vectors of a two-dimensional plane:



And sum of two complex numbers = sum of their corresponding vectors.

- A product of two complex numbers is again a complex number:

$$(a_1+b_1i)\cdot(a_2+b_2i) = a_1a_2+a_1b_2i+a_2b_1i+b_1b_2\underbrace{i^2}_{=-1} =$$

$$= (a_1a_2 - b_1b_2) + (a_1b_2 + a_2b_1)i$$

- Euler's formula: $e^{i\varphi} = \cos\varphi + i\sin\varphi$. If you want to check it, just write Taylor's expansions of exponent, sine and cosine and use the fact that $i^2 = -1$, $i^3 = -i$, $i^4 = 1$, etc.

- Multiplication by $e^{i\varphi}$ rotates a vector by angle φ (counterclockwise if $\varphi > 0$):

$$e^{i\varphi}z = (\cos\varphi + i\sin\varphi)(a+bi) =$$

$$(a\cos\varphi - b\sin\varphi) + (b\cos\varphi + a\sin\varphi) =$$

$$\begin{pmatrix} a & b \end{pmatrix} \cdot \begin{pmatrix} \cos\varphi & \sin\varphi \\ -\sin\varphi & \cos\varphi \end{pmatrix}$$

which is indeed a result of rotation of $z = (a, b)$ by angle φ.